

Patient Churn Prediction Project

AUTHOR

Sam Brothers, Pooja Muthuraj, Alex Farina, Ashley Gasswint

PUBLISHED

December 3, 2025

TEAM SUMMARY

The model confirms that churn is primarily driven by life stage and socioeconomic instability, not by clinical or appointment-keeping behavior alone.

The difference between the highest-risk group (Age 65+) and the lowest-risk group (Spanish/Other Language speakers) spans a massive range, from a 74.9% churn likelihood down to a 6.7% likelihood.

Top Churn Drivers:

1. Extreme Age Risk

- Age is the dominant predictor. The jump from the baseline (18-19) to the 65+ group (risk of 74.9%) is quite severe
- Could necessitates specialized senior care engagement to address mobility, health literacy, and social isolation barriers.

2. Socioeconomic Instability

- Housing and employment uncertainty double the odds of churn. Patients who are Homeless, Lives With Others, Self Employed, or Unemployed are struggling with basic needs, making their clinic appointments or overall a lower priority over other needs.

Top Retention Factors:

1. Language Services Success

- Patients speaking Spanish or "Other" languages are highly retained. Their churn risk drops significantly (~7% to 8% absolute risk).

2. Chronic Care Engagement

- Patients with a Chronic Diagnosis are less likely to churn. Their ongoing health needs may foster stronger clinic relationships and appointment adherence.

1. SETUP

1.1 Load Packages

```
library(tidyverse)
```

Warning: package 'ggplot2' was built under R version 4.3.3

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.1.4      ✓ readr      2.1.4
✓ forcats    1.0.0      ✓ stringr    1.5.0
✓ ggplot2    3.5.2      ✓ tibble     3.2.1
✓ lubridate  1.9.2      ✓ tidyr      1.3.0
✓ purrr      1.0.2

— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
library(fastDummies)
```

Warning: package 'fastDummies' was built under R version 4.3.3

```
library(ISLR2)
```

Warning: package 'ISLR2' was built under R version 4.3.3

```
library(class)
library(zipcodeR)
library(geosphere)
```

Warning: package 'geosphere' was built under R version 4.3.3

```
library(pROC)
```

Warning: package 'pROC' was built under R version 4.3.3

Type 'citation("pROC")' for a citation.

Attaching package: 'pROC'

The following objects are masked from 'package:stats':

cov, smooth, var

```
library(caret)
```

Warning: package 'caret' was built under R version 4.3.3

```
Loading required package: lattice
Registered S3 method overwritten by 'future':
  method          from
all.equal.connection parallelly
```

Attaching package: 'caret'

The following object is masked from 'package:purrr':

```
lift
```

```
library(MASS)
```

Attaching package: 'MASS'

The following object is masked from 'package:ISLR2':

```
Boston
```

The following object is masked from 'package:dplyr':

```
select
```

1.2 Load data

```
people <- read_csv("people.csv", show_col_types = FALSE)
encounters <- read_csv("encounters.csv", show_col_types = FALSE)
enrollment <- read_csv("enrollment.csv", show_col_types = FALSE)
noshow <- read_csv("noshow.csv", show_col_types = FALSE)
observations <- read_csv("observations.csv", show_col_types = FALSE)
diagnosis <- read_csv("diagnosis.csv", show_col_types = FALSE)
orders <- read_csv("orders.csv", show_col_types = FALSE)
residence <- read_csv("residence.csv", show_col_types = FALSE)
```

1.3 Set Reference Date for Reproducibility

CRITICAL: Use fixed date (when data was received), NOT Sys.Date() This ensures everyone gets the same results.

```
reference_date <- as.Date("2025-09-30") # End of September 2025

cat(sprintf("%17s Reference date: %s (data receipt/extraction date)\n",
            reference_date))
```

```
%17s Reference date: 2025-09-30 (data receipt/extraction date)
```

```
cat("    All 'days since' calculations will use this date.\n\n")
```

All 'days since' calculations will use this date.

2. DEFINE THE CHURN VARIABLE

CRITICAL: Choose ONE approach and document your rationale Goal: CHURN = (No Completed Encounter in 365 days) OR (Disenrolled Status)

2.1 Identify the encounter date column dynamically

```
enc_date_col <- names(encounters)[grepl("date|datetime|time", names(encounters),  
                                         ignore.case = TRUE)][1]
```

2.2: Encounter-Based Churn (365-day Gap)

- CRITICAL: Convert to Date format
- CRITICAL: Filter for only 'Completed' encounters (Active care)
- Filter out data that occurred AFTER our analysis date (no look-ahead)
- FLAG 1: Churned if last completed encounter was more than 365 days ago

```
churn_encounter_flag <- encounters %>%  
  mutate(enc_date = as.Date(.data[[enc_date_col]])) %>%  
  filter(enc_status == "Completed") %>%  
  filter(enc_date <= reference_date) %>%  
  group_by(mrn_pseudo) %>%  
  summarize(  
    last_completed_encounter = max(enc_date, na.rm = TRUE),  
    days_since_completed_enc = as.numeric(  
      reference_date - last_completed_encounter),  
    .groups = "drop"  
  ) %>%  
  mutate(  
    churn_enc_flag = if_else(days_since_completed_enc > 365, 1, 0, missing = 1)  
  )
```

2.3: Final Fix

Select the columns outside the pipe using Base R indexing

```
churn_encounter_flag <- churn_encounter_flag[, c("mrn_pseudo", "churn_enc_flag")]
```

2.4: Enrollment-Based Churn (Disenrolled Status)

- Filter for records that explicitly mark the patient as 'Disenrolled'
- Only count disenrolled status if the event occurred BEFORE or AT the reference date
- FLAG 2: Churned if the patient has ever been formally disenrolled

```
churn_disenrolled_flag <- enrollment %>%
  filter(enr_status == "Disenrolled") %>%
  filter(as.Date(enr_start) <= reference_date) %>%
  group_by(mrn_pseudo) %>%
  summarize(
    churn_disenr_flag = 1,
    .groups = "drop"
  )
```

2.5: Removing the simple churn_encounter object from the old code if it still exists

```
if(exists("churn_encounter")) rm(churn_encounter)
```

3. CREATE PATIENT-LEVEL SUMMARIES

3.1: Enrollment summary

- Use if() instead of if_else() to avoid warnings

```
if ("enr_start" %in% names(enrollment) && "enr_exp" %in% names(enrollment)) {
  enrollment_summary <- enrollment %>%
    filter(enr_start <= enr_exp | is.na(enr_start) | is.na(enr_exp)) %>%
    group_by(mrn_pseudo) %>%
    summarize(
      total_enrollments = n(),
      first_enrollment = if(all(is.na(enr_start))) as.Date(NA_character_)
      else min(enr_start, na.rm = TRUE),
      last_enrollment_end = if(all(is.na(enr_exp))) as.Date(NA_character_)
      else max(enr_exp, na.rm = TRUE),
      total_enrollment_days = sum(as.numeric(enr_exp - enr_start), na.rm = TRUE),
      currently_enrolled = if(all(is.na(enr_exp)))
        FALSE else max(enr_exp, na.rm = TRUE) >= reference_date,
      .groups = "drop"
    )
}
```

3.2: Encounters summary

- Handles different date column names

```
enc_date_col <- names(encounters)[grepl("date|datetime|time",
                                         names(encounters), ignore.case = TRUE)]

if (length(enc_date_col) > 0 && inherits(encounters[[enc_date_col[1]]],
                                         c("Date", "POSIXct", "POSIXt"))) {
  enc_date_col <- enc_date_col[1]

  if ("telehealth" %in% names(encounters)) {
    encounters_summary <- encounters %>%
      group_by(mrn_pseudo) %>%
      summarize(
        total_encounters = n(),
        telehealth_count = sum(telehealth == "Yes", na.rm = TRUE),
        telehealth_pct = mean(telehealth == "Yes", na.rm = TRUE) * 100,
        first_visit = if(all(is.na(.data[[enc_date_col]])))
          as.Date(NA_character_) else min(.data[[enc_date_col]], na.rm = TRUE),
        last_visit = if(all(is.na(.data[[enc_date_col]])))
          as.Date(NA_character_) else max(.data[[enc_date_col]], na.rm = TRUE),
        days_since_last_visit = if(all(is.na(.data[[enc_date_col]])))
          NA_real_ else as.numeric(reference_date - max(.data[[enc_date_col]],
                                                         na.rm = TRUE)),
        visits_in_past_year = sum(
          !is.na(.data[[enc_date_col]]) &
            .data[[enc_date_col]] >= (reference_date - 365) &
            .data[[enc_date_col]] <= reference_date
        ),
        .groups = "drop"
      )
  } else if ("enc_is_teleh" %in% names(encounters)) {
    # enc_is_teleh = 0/1
    encounters_summary <- encounters %>%
      group_by(mrn_pseudo) %>%
      summarize(
        total_encounters = n(),
        telehealth_count = sum(enc_is_teleh == 1, na.rm = TRUE),
        telehealth_pct = mean(enc_is_teleh == 1, na.rm = TRUE) * 100,
        first_visit = if(all(is.na(.data[[enc_date_col]])))
          as.Date(NA_character_) else min(.data[[enc_date_col]], na.rm = TRUE),
        last_visit = if(all(is.na(.data[[enc_date_col]])))
          as.Date(NA_character_) else max(.data[[enc_date_col]], na.rm = TRUE),
        days_since_last_visit = if(all(is.na(.data[[enc_date_col]])))
          NA_real_ else as.numeric(reference_date - max(.data[[enc_date_col]],
                                                         na.rm = TRUE)),
        visits_in_past_year = sum(
          !is.na(.data[[enc_date_col]]) &
            .data[[enc_date_col]] >= (reference_date - 365) &
```

```

        .data[[enc_date_col]] <= reference_date
      ),
      .groups = "drop"
    )
  } else {
    encounters_summary <- encounters %>%
      group_by(mrn_pseudo) %>%
      summarize(
        total_encounters = n(),
        first_visit = if(all(is.na(.data[[enc_date_col]])))
          as.Date(NA_character_) else min(.data[[enc_date_col]], na.rm = TRUE),
        last_visit = if(all(is.na(.data[[enc_date_col]])))
          as.Date(NA_character_) else max(.data[[enc_date_col]], na.rm = TRUE),
        days_since_last_visit = if(all(is.na(.data[[enc_date_col]])))
          NA_real_ else as.numeric(reference_date - max(.data[[enc_date_col]],
                                                         na.rm = TRUE)),
        visits_in_past_year = sum(
          !is.na(.data[[enc_date_col]]) &
            .data[[enc_date_col]] >= (reference_date - 365) &
            .data[[enc_date_col]] <= reference_date
        ),
        .groups = "drop"
      )
  }
}

```

3.3: No shows summary

```

noshow_summary <- noshow %>%
  group_by(mrn_pseudo) %>%
  summarize(
    total_noshows = n(),
    .groups = "drop"
  )

```

3.4: Add no show proportion

```

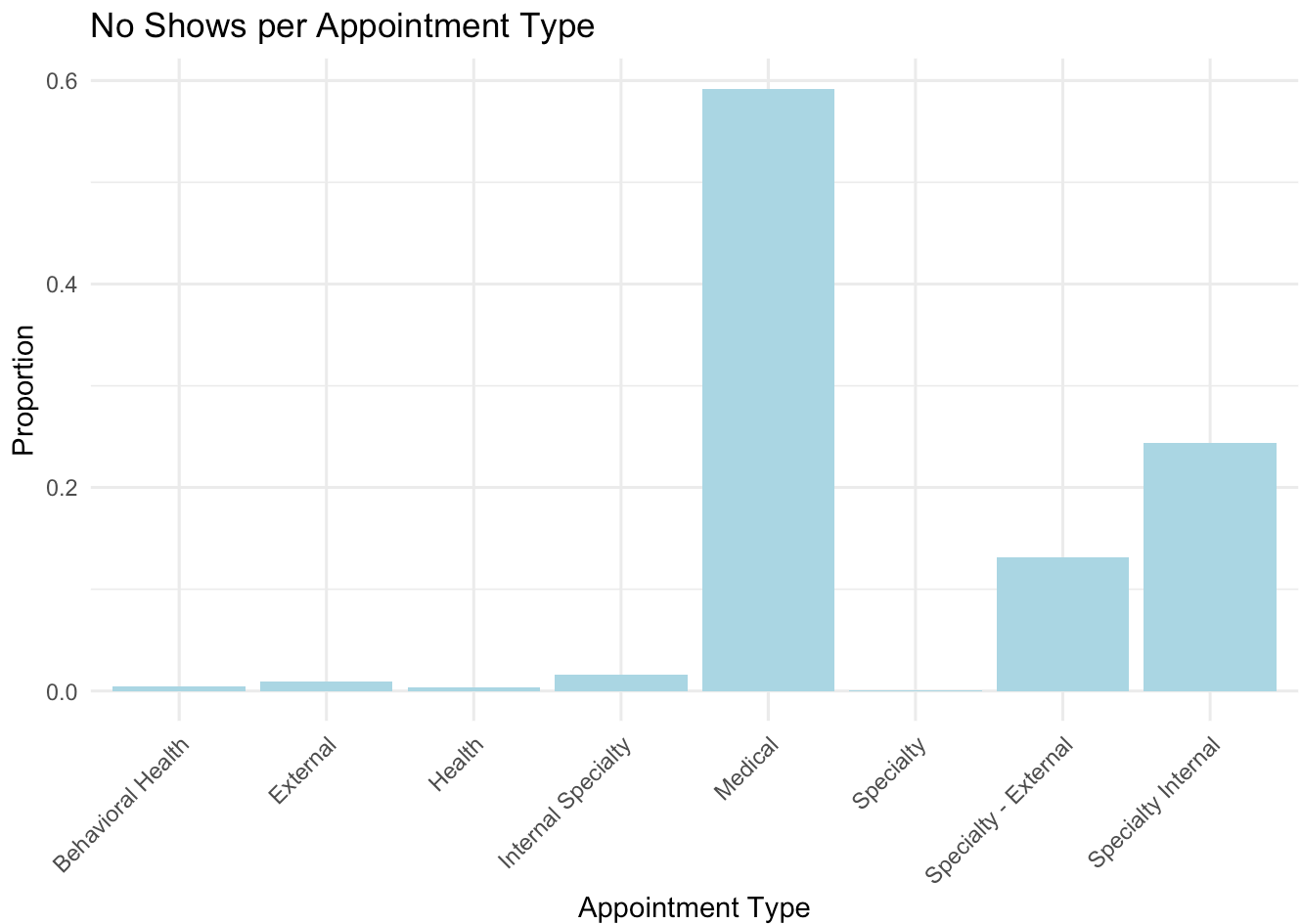
noshow <- noshow %>%
  mutate(ns_reason = str_to_title(str_to_lower(ns_reason)))

noshow <- noshow %>%
  filter(ns_reason != "Dental")

ggplot(data = noshow, aes(x = ns_reason, y = after_stat(prop), group = 1)) +
  geom_bar(fill = "lightblue") +
  labs(title = "No Shows per Appointment Type", x = "Appointment Type",
       y = "Proportion") +

```

```
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



4. BUILD MASTER DATASET (ONE ROW PER PATIENT)

```
master <- people %>%
  # Join summaries
  left_join(enrollment_summary, by = "mrn_pseudo") %>%
  left_join(encounters_summary, by = "mrn_pseudo") %>%
  left_join(noshows_summary, by = "mrn_pseudo") %>%
  # Create binary flags for sparse data
  mutate(
    has_observations = mrn_pseudo %in% observations$mrn_pseudo,
    has_diagnosis = mrn_pseudo %in% diagnosis$mrn_pseudo,
    has_noshows = !is.na(total_noshows)
  )

# Save raw version for missing data analysis
master_raw <- master

# Add chosen churn definition
master <- master %>%
```



```
left_join(churn_encounter_flag, by = "mrn_pseudo") %>%
left_join(churn_disenrolled_flag, by = "mrn_pseudo")
```

5. MISSING DATA ANALYSIS & HANDLING

```
cat("\n=== MISSING DATA SUMMARY (RAW DATA) ===\n")
```

```
=== MISSING DATA SUMMARY (RAW DATA) ===
```

```
# Calculate % missing for key variables (use RAW data)
missing_summary <- master_raw %>%
  summarize(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "n_missing") %>%
  mutate(pct_missing = (n_missing / nrow(master_raw)) * 100) %>%
  filter(pct_missing > 0) %>%
  arrange(desc(pct_missing))

cat(sprintf("Variables with missing data: %d\n", nrow(missing_summary)))
```

```
Variables with missing data: 20
```

```
if (nrow(missing_summary) > 0) {
  cat(sprintf("Most missing: %s (%.1f%%)\n\n",
    missing_summary$variable[1],
    missing_summary$pct_missing[1]))
}
```

```
Most missing: total_noshows (90.0%)
```

```
# STRATEGY: Domain-specific defaults
# For Lackey data, most NAs are structural (patient never had that event)

master <- master %>%
  mutate(
    # Counts: NA means zero occurrences
    total_encounters = replace_na(total_encounters, 0),
    total_noshows = replace_na(total_noshows, 0),
    total_enrollments = replace_na(total_enrollments, 0),

    # Rates/percentages: NA means 0% (or could keep as NA)
    telehealth_pct = replace_na(telehealth_pct, 0),

    # Time features: NA means "never happened" – use sentinel value
    days_since_last_visit = replace_na(days_since_last_visit, 9999),

    # Create indicators for structural missingness (informative!)
```

```

    never_visited = is.na(first_visit),
    never_enrolled = is.na(first_enrollment)
  )

cat("✓ Applied domain-specific missing data handling\n")

```

✓ Applied domain-specific missing data handling

```
cat("  • Counts → 0 (no events recorded)\n")
```

- Counts → 0 (no events recorded)

```
cat("  • Rates → 0% (no activity)\n")
```

- Rates → 0% (no activity)

```
cat("  • Time features → 9999 days (sentinel value for 'never')\n")
```

- Time features → 9999 days (sentinel value for 'never')

```
cat("  • Created 'never_*' indicators\n\n")
```

- Created 'never_*' indicators

```

# Validation: Check remaining NAs
remaining_na <- master %>%
  summarize(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything()) %>%
  filter(value > 0)

if (nrow(remaining_na) > 0) {
  cat(sprintf("Note: %d variables still have NAs (dates, demographics)\n",
              nrow(remaining_na)))
  cat("    This is expected – will derive features from dates in Part 6\n")
  cat("    Time features (days_since_last_visit) already handled with sentinel
    value\n\n")
} else {
  cat("✓ No remaining NAs in dataset\n\n")
}

```

Note: 17 variables still have NAs (dates, demographics)

This is expected – will derive features from dates in Part 6

Time features (days_since_last_visit) already handled with sentinel value

6. FEATURE ENGINEERING

6.1: Engineering part 1

```
master <- master %>%
  mutate(
    # Transformations (for skewed distributions)
    log_encounters = log(total_encounters + 1), # +1 handles zeros

    # Ratios & Rates (handle division by zero)
    noshow_rate = if_else(total_encounters > 0,
                          total_noshows / (total_noshows + total_encounters),
                          NA_real_),

    # Visit frequency (encounters per year of enrollment)
    visit_frequency = if_else(total_enrollment_days > 0,
                              (total_encounters / total_enrollment_days) * 365,
                              NA_real_),

    # Temporal features
    days_since_exp = if_else(!is.na(last_enrollment_end),
                             as.numeric(reference_date - last_enrollment_end),
                             NA_real_),

    tenure_days = if_else(!is.na(first_enrollment) & !is.na(last_enrollment_end),
                          as.numeric(last_enrollment_end - first_enrollment),
                          NA_real_)
  )

## drop all the family members (never_enrolled = TRUE)
master_new <- master %>% filter(never_enrolled == FALSE)
```

6.2: Diagnosis code grouping examples

- All the diagnoses with labels are considered chronic

```
diagnosis <- diagnosis %>% mutate(chronic =
                                case_when(dia_dxgroup == "Other" ~ 0,
                                            TRUE ~ 1))

# join into master_new
diagnosis_summary <- diagnosis %>%
  group_by(mrn_pseudo) %>%
  summarize(
    has_chronic_diagnosis = max(chronic, na.rm = TRUE),
    .groups = 'drop'
  )

master_new <- master_new %>%
  left_join(diagnosis_summary, by = "mrn_pseudo") %>%
```

```
mutate(
  has_chronic_diagnosis = replace_na(has_chronic_diagnosis, 0)
)

## drop all the family members (never_enrolled = TRUE)
master_new <- master_new %>% filter(never_enrolled == FALSE)
```

6.3: Final Churn Variable Combination

- This replaces the old complex case_when logic
- Fill NA churn flags with 0.
- This correctly handles patients who might not have an encounter/enrollment record, but were not deemed 'churned' by the initial flag creation (e.g., they exist in 'people' but nowhere else).

```
master_new <- master_new %>%

mutate(
  churn_enc_flag = replace_na(churn_enc_flag, 0),
  churn_disenr_flag = replace_na(churn_disenr_flag, 0)
) %>%

# FINAL CHURN DEFINITION: CHURN = (Encounter Gap > 365d) OR (Disenrolled Status)
mutate(
  # The pmax() function performs a logical OR: Churned (1) if EITHER flag is 1.
  churned = pmax(churn_enc_flag, churn_disenr_flag),
  # Convert to a clear factor for modeling
  churned = factor(churned, levels = c(0, 1))#, labels = c("Retained", "Churned"))
)

master_new <- master_new[, !names(master_new) %in% c("churn_enc_flag",
  "churn_disenr_flag")]

# Drop the temporary flags

table(master_new$churned)
```

```

0      1
3735 2881
```

7. DATA QUALITY CHECKS

```
cat("\n=== FINAL DATA QUALITY CHECKS ===\n")
```

```
=== FINAL DATA QUALITY CHECKS ===
```

```
# Check for missing outcome on the master_new dataset
if ("churned" %in% names(master_new)) {
  cat(sprintf("Missing churn labels: %d (%.1f%%)\n",
              sum(is.na(master_new$churned)),
              mean(is.na(master_new$churned)) * 100))

  # Check class balance
  master_new %>%
    filter(!is.na(churned)) %>%
    count(churned) %>%
    mutate(percentage = n / sum(n) * 100) %>%
    print()
} else {
  cat("⚠️ No churn variable found – remember to add it!\n")
}
```

Missing churn labels: 0 (0.0%)

A tibble: 2 × 3

	churned	n	percentage
	<fct>	<int>	<dbl>
1	0	3735	56.5
2	1	2881	43.5

```
# Complete cases comparison
cat(sprintf("\nComplete cases (CLEANED): %s / %s (%.1f%%)\n",
            format(sum(complete.cases(master_new)), big.mark = ","),
            format(nrow(master_new), big.mark = ","),
            (sum(complete.cases(master_new)) / nrow(master_new)) * 100))
```

Complete cases (CLEANED): 4,068 / 6,616 (61.5%)

```
cat(sprintf("Complete cases (CLEANED): %s / %s (%.1f%%)\n",
            format(sum(complete.cases(master)), big.mark = ","),
            format(nrow(master), big.mark = ","),
            (sum(complete.cases(master)) / nrow(master)) * 100))
```

Complete cases (CLEANED): 1,024 / 15,606 (6.6%)

```
cat("\n🌟 Script complete! Master dataset ready for modeling.\n")
```

🌟 Script complete! Master dataset ready for modeling.

```
cat(sprintf("📅 Remember: All time calculations use reference_date = %s\n",
            reference_date))
```

📅 Remember: All time calculations use reference_date = 2025-09-30

```

age_levels <- c("Under 18", "18-19", "20-29", "30-39",
               "40-49", "50-59", "60-64", "65+")
master_new <- master_new %>%
  mutate(
    age_range = factor(
      age_range,
      levels = age_levels,
      ordered = FALSE
    )
  )

master_new <- master_new %>%
  mutate(
    Employment = str_to_title(Employment),
    Employment = str_replace_all(Employment, "\\]", "("),
    Employment = if_else(Employment %in% c("Null", "Unknown"),
                          NA_character_, Employment),
    Employment = factor(
      Employment,
      levels = c(
        "Employed", "Self Employed", "Seasonal",
        "Unemployed (Actively Looking For Work)",
        "Not In Workforce (Choosing Not To Work)", "Retired"
      )
    )
  )

master_new <- master_new %>% mutate(Education = factor(
  Education))

master_new <- master_new %>%
  mutate(
    Housing = str_to_title(Housing), # make capitalization consistent
    Housing = if_else(
      Housing %in% c("Null", "Unknown", ""),
      NA_character_,
      Housing
    ),
    Housing = factor(
      Housing,
      levels = c(
        "Own",
        "Rent",
        "Lives With Others",
        "Mobile Home",
        "Homeless"
      )
    )
  )

```

```

master_new <- master_new %>%
  mutate(
    Language = str_to_title(Language),
    Language = str_trim(Language),
    Language = str_replace_all(Language, "Other:", ""),
    Language = str_replace_all(Language, "Other", ""),
    Language = str_replace_all(Language, ":", ""),
    Language = if_else(
      Language %in% c("Null", "Unknown", "", NA_character_),
      NA_character_,
      Language
    ),

    # Collapse into 4 buckets
    Language_grouped = case_when(
      str_detect(Language, "English") ~ "English",
      str_detect(Language, "Spanish") ~ "Spanish",
      is.na(Language) ~ "Unknown/Missing",
      TRUE ~ "Other"
    ),

    # Convert to factor with clear levels
    Language_grouped = factor(
      Language_grouped,
      levels = c("English", "Spanish", "Other", "Unknown/Missing")
    )
  )

master_new <- master_new %>%
  mutate(
    has_chronic_diagnosis = factor(
      has_chronic_diagnosis
    )
  )

```

8. FEATURE TYPES

8.1: Demographic Features

8.11 Language

- Patients whose language differs from English, what is most likely to be spoken at lackey, may experience more difficulty with their healthcare than others, increasing chances for churn.
- Data Issues: Way too many groups and varying notation of language spoken, so we decided to group them into 4 main categories

```
language_raw_summary <- master_new %>%
  summarise(
    total = n(),
    missing = sum(is.na(Language)),
    missing_pct = mean(is.na(Language)) * 100
  )

language_raw_counts <- master_new %>%
  count(Language) %>%
  arrange(desc(n)) %>%
  mutate(percent = n / sum(n, na.rm = TRUE) * 100)

language_raw_summary
```

```
# A tibble: 1 × 3
  total missing missing_pct
  <int>   <int>       <dbl>
1  6616     207         3.13
```

```
print(language_raw_counts, n = 39)
```

```
# A tibble: 39 × 3
  Language                n percent
  <chr>                <int>   <dbl>
1 "English"            5104  77.1
2 "Spanish"            1100  16.6
3  <NA>                 207   3.13
4 "Arabic"              46   0.695
5 "Korean"              36   0.544
6 "Chinese"             31   0.469
7 "Farsi"              19   0.287
8 "Hindi"              12   0.181
9 " Russian"            9   0.136
10 " French"            5   0.0756
11 " Nepali"            3   0.0453
12 " Portuguese"       3   0.0453
13 " Vietnamese"       3   0.0453
14 "Urdu"               3   0.0453
15 " "                  2   0.0302
16 " Bangla"           2   0.0302
17 " Creole"           2   0.0302
18 " Dari"             2   0.0302
19 " English"          2   0.0302
20 " Filipino"         2   0.0302
21 " Igbo"             2   0.0302
22 " Mongolian"        2   0.0302
23 " Tagalog"          2   0.0302
24 " Ukrainian"        2   0.0302
25 " Bangli"           1   0.0151
```


26	" Broken English"	1	0.0151
27	" Haitian Creole"	1	0.0151
28	" Hungarian"	1	0.0151
29	" Pashto"	1	0.0151
30	" Português"	1	0.0151
31	" Russia"	1	0.0151
32	" Russian / Ukranian"	1	0.0151
33	" Russian, Ukrainian"	1	0.0151
34	" Russian/Ukrainian"	1	0.0151
35	" Tagalog Filipino"	1	0.0151
36	" Turkish"	1	0.0151
37	" Twi (Ghana)"	1	0.0151
38	"Vietnamese"	1	0.0151
39	"mongolian"	1	0.0151

```
lang_summary <- master_new %>%
  summarise(
    total = n(),
    missing = sum(is.na(Language_grouped)),
    missing_pct = mean(is.na(Language_grouped)) * 100
  )
```

```
lang_counts <- master_new %>%
  count(Language_grouped) %>%
  mutate(percent = n / sum(n) * 100)
```

```
lang_summary
```

```
# A tibble: 1 × 3
  total missing missing_pct
<int>   <int>       <dbl>
1  6616     0         0
```

```
lang_counts
```

```
# A tibble: 4 × 3
  Language_grouped     n percent
<fct>           <int>   <dbl>
1 English         5107   77.2
2 Spanish         1100   16.6
3 Other           202    3.05
4 Unknown/Missing  207    3.13
```

8.12: Housing Feature

- Unstable housing conditions could face more issues around mobility and overall stress

```
housing_summary <- master_new %>%
  summarise(
```

```

    total = n(),
    missing = sum(is.na(Housing)),
    missing_pct = mean(is.na(Housing)) * 100
  )

housing_counts <- master_new %>%
  count(Housing) %>%
  mutate(percent = n / sum(n) * 100)

housing_summary

```

```

# A tibble: 1 × 3
  total missing missing_pct
<int>   <int>      <dbl>
1  6616     35      0.529

```

```
housing_counts
```

```

# A tibble: 6 × 3
  Housing          n percent
<fct>      <int>   <dbl>
1 Own           911  13.8
2 Rent        3334  50.4
3 Lives With Others 2121  32.1
4 Mobile Home    135   2.04
5 Homeless       80   1.21
6 <NA>          35   0.529

```

8.13: Employment Feature

- Those unemployed may be in the process of gaining medicare/medicaid and those employed may be too busy to make it to appointments
- Data Issues: Some small issues with ('s and ['s in notation

```

employment_summary <- master_new %>%
  summarise(
    total = n(),
    missing = sum(is.na(Employment)),
    missing_pct = mean(is.na(Employment)) * 100
  )

employment_counts <- master_new %>%
  count(Employment) %>%
  mutate(percent = n / sum(n) * 100)

employment_summary

```

```

# A tibble: 1 × 3
  total missing missing_pct

```

	<int>	<int>	<dbl>
1	6616	48	0.726

```
employment_counts
```

```
# A tibble: 7 × 3
```

	Employment <fct>	n <int>	percent <dbl>
1	Employed	3524	53.3
2	Self Employed	381	5.76
3	Seasonal	82	1.24
4	Unemployed (Actively Looking For Work)	2027	30.6
5	Not In Workforce (Choosing Not To Work)	209	3.16
6	Retired	345	5.21
7	<NA>	48	0.726

8.14: Zip Code Distance Feature

- Patients who live closer to lackey may be more likely to visit and less likely to churn. We will calculate distance from the geographical center of each county to lackey's lat and long in meter.

```
ref_lat <- 37.23110133397947
ref_lng <- -76.55492831279969

residence <- residence %>%
  group_by(mrn_pseudo) %>%
  slice_max(order_by = r_start, n=1, with_ties = FALSE) %>%
  ungroup()

residence$r_zip <- as.character(residence$r_zip)

# Add latitude and longitude for each ZIP
zip_lookup <- zipcodeR::zip_code_db[, c("zipcode", "lat", "lng")]
colnames(zip_lookup)
```

```
[1] "zipcode" "lat"      "lng"
```

```
colnames(zip_lookup)[1] <- "r_zip"

residence <- merge(residence, zip_lookup, by = "r_zip", all.x = TRUE)

# Compute distance for each ZIP
residence$distance_meters <- distHaversine(
  cbind(residence$lng, residence$lat),
  c(ref_lng, ref_lat)
)

# cleaning and joining
residence_clean <- residence %>%
```

```

drop_na(distance_meters)

residence_dist <- residence_clean

residence_dist <- residence_dist[c("mrn_pseudo", "distance_meters")]

master_new <- master_new %>%
  left_join(residence_dist, by = "mrn_pseudo")

```

8.15 No-Show Rate Feature Summary

- Patients with higher no-show rates may be more likely to churn, since missed appointments often indicate lower engagement or access barriers.

```

noshow_summary_stats <- master_new %>%
  summarise(
    total = n(),
    missing = sum(is.na(noshow_rate)),
    missing_pct = mean(is.na(noshow_rate)) * 100,
    mean_noshow_rate = mean(noshow_rate, na.rm = TRUE),
    median_noshow_rate = median(noshow_rate, na.rm = TRUE),
    min_noshow_rate = min(noshow_rate, na.rm = TRUE),
    max_noshow_rate = max(noshow_rate, na.rm = TRUE)
  )

noshow_summary_stats

```

```

# A tibble: 1 × 7
  total missing missing_pct mean_noshow_rate median_noshow_rate min_noshow_rate
<int>   <int>      <dbl>          <dbl>              <dbl>              <dbl>
1  6616    2519      38.1            0.0224              0                  0
# i 1 more variable: max_noshow_rate <dbl>

```

```

# Summary statistics for has_chronic_condition
chronic_summary <- master_new %>%
  summarise(
    total = n(),
    missing = sum(is.na(has_chronic_diagnosis)),
    missing_pct = mean(is.na(has_chronic_diagnosis)) * 100
  )

# Frequency table
chronic_counts <- master_new %>%
  count(has_chronic_diagnosis) %>%
  mutate(percent = n / sum(n) * 100)

chronic_summary

```

```
# A tibble: 1 × 3
  total missing missing_pct
<int>   <int>       <dbl>
1  6616       0         0
```

```
chronic_counts
```

```
# A tibble: 2 × 3
  has_chronic_diagnosis    n percent
<fct>             <int>   <dbl>
1 0                 5178    78.3
2 1                 1438    21.7
```

9. CREATING TRAIN AND TEST DATA

```
set.seed(123) # for reproducibility

# 70% training indices
train_indices <- sample(seq_len(nrow(master_new)), size = 0.7 * nrow(master_new))

# Split
train_data <- master_new[train_indices, ]
test_data  <- master_new[-train_indices, ]
```

10. CREATING THE MODEL

```
churn_model <- glm(
  churned ~ Employment + has_chronic_diagnosis + noshow_rate + age_range +
    Housing + Language_grouped + distance_meters,
  data = train_data,
  family = binomial)

summary(churn_model)
```

Call:

```
glm(formula = churned ~ Employment + has_chronic_diagnosis +
     noshow_rate + age_range + Housing + Language_grouped + distance_meters,
     family = binomial, data = train_data)
```

Coefficients:

	Estimate	Std. Error	z value
(Intercept)	-1.047e+00	7.308e-01	-1.433
EmploymentSelf Employed	5.833e-01	1.872e-01	3.115
EmploymentSeasonal	2.185e-02	4.068e-01	0.054
EmploymentUnemployed (Actively Looking For Work)	4.794e-01	1.083e-01	4.428

EmploymentNot In Workforce (Choosing Not To Work)	2.451e-01	2.368e-01	1.035
EmploymentRetired	3.350e-01	2.674e-01	1.253
has_chronic_diagnosis1	-3.419e-01	8.958e-02	-3.817
noshow_rate	-4.642e-01	6.672e-01	-0.696
age_range20-29	1.641e+00	7.244e-01	2.265
age_range30-39	1.910e+00	7.229e-01	2.642
age_range40-49	1.912e+00	7.241e-01	2.640
age_range50-59	1.969e+00	7.235e-01	2.722
age_range60-64	1.813e+00	7.270e-01	2.494
age_range65+	2.724e+00	7.300e-01	3.732
HousingRent	-3.965e-03	1.287e-01	-0.031
HousingLives With Others	2.888e-01	1.435e-01	2.013
HousingMobile Home	-4.915e-01	2.849e-01	-1.725
HousingHomeless	-2.168e-01	4.182e-01	-0.518
Language_groupedSpanish	-7.533e-01	1.068e-01	-7.054
Language_groupedOther	-8.149e-01	2.234e-01	-3.648
Language_groupedUnknown/Missing	3.429e-01	2.926e-01	1.172
distance_meters	-1.229e-05	4.693e-06	-2.619

Pr(>|z|)

(Intercept)	0.151957
EmploymentSelf Employed	0.001838 **
EmploymentSeasonal	0.957166
EmploymentUnemployed (Actively Looking For Work)	9.53e-06 ***
EmploymentNot In Workforce (Choosing Not To Work)	0.300612
EmploymentRetired	0.210230
has_chronic_diagnosis1	0.000135 ***
noshow_rate	0.486557
age_range20-29	0.023526 *
age_range30-39	0.008232 **
age_range40-49	0.008289 **
age_range50-59	0.006491 **
age_range60-64	0.012618 *
age_range65+	0.000190 ***
HousingRent	0.975423
HousingLives With Others	0.044147 *
HousingMobile Home	0.084533 .
HousingHomeless	0.604198
Language_groupedSpanish	1.74e-12 ***
Language_groupedOther	0.000265 ***
Language_groupedUnknown/Missing	0.241261
distance_meters	0.008815 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 3542.3 on 2747 degrees of freedom

Residual deviance: 3308.1 on 2726 degrees of freedom

(1883 observations deleted due to missingness)

AIC: 3352.1

Number of Fisher Scoring iterations: 4

```
# 1. Inspect the levels the model actually learned (optional, but informative)
print(churn_model$xlevels$age_range)
```

```
[1] "18-19" "20-29" "30-39" "40-49" "50-59" "60-64" "65+"
```

```
# 2. Force the factor levels in test_data to match the model's learned levels
test_data$age_range <- factor(
  test_data$age_range,
  levels = churn_model$xlevels$age_range # Use the levels stored in the trained
  #model
)

# Predicted probabilities
test_data$predicted_prob <- predict(churn_model, newdata = test_data,
                                   type = "response")

# Predicted classes using 0.5 threshold
test_data <- test_data %>%
  mutate(predicted_churn = if_else(predicted_prob > 0.6, 1, 0))

# Confusion matrix
table(Predicted = test_data$predicted_churn, Actual = test_data$churned)
```

	Actual	
Predicted	0	1
0	179	226
1	206	612

```
# Calculate accuracy
mean(test_data$predicted_churn == test_data$churned, na.rm = TRUE)
```

```
[1] 0.6467702
```

```
roc_obj <- roc(test_data$churned, test_data$predicted_prob)
```

Setting levels: control = 0, case = 1

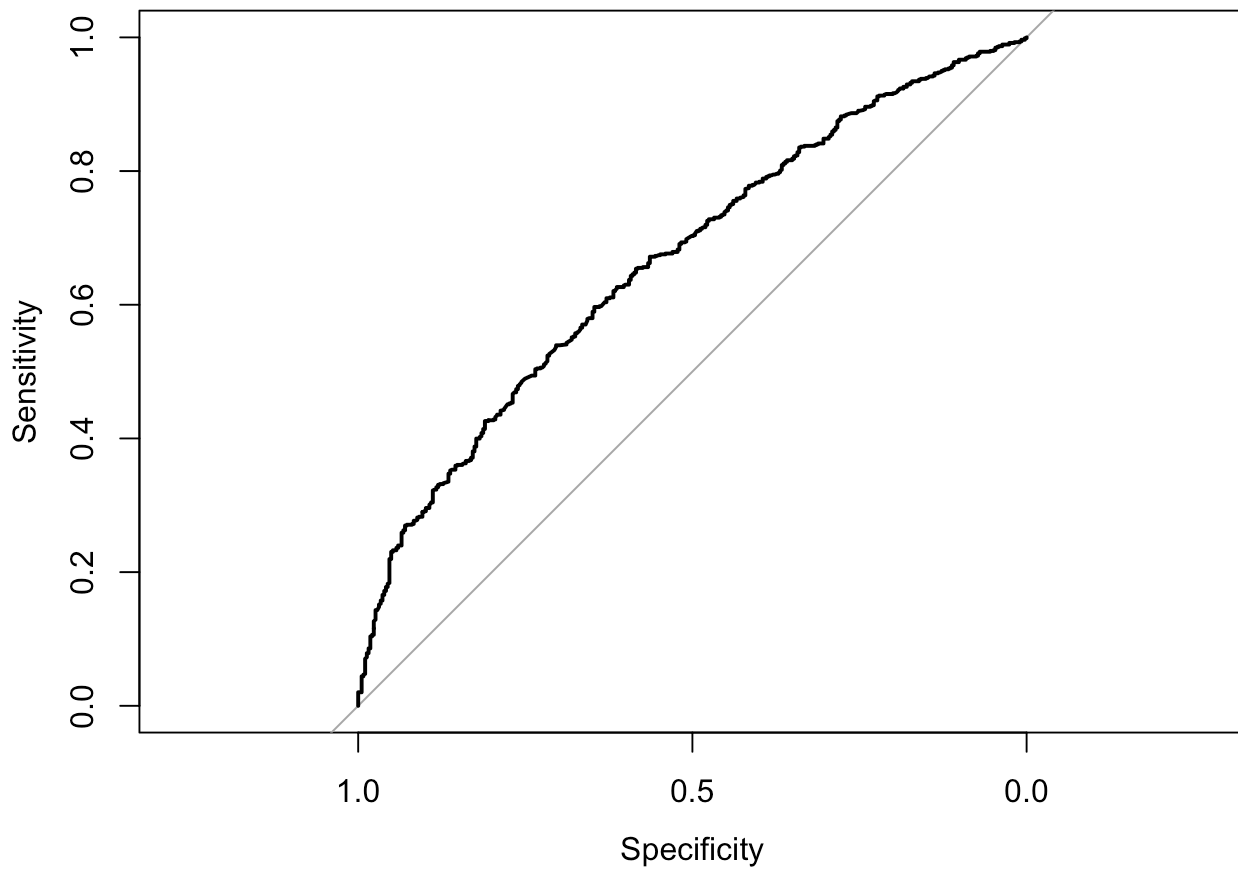
Setting direction: controls < cases

```
auc(roc_obj)
```

Area under the curve: 0.6661

```
plot(roc_obj, main = "ROC Curve for Churn Model")
```

ROC Curve for Churn Model



```
table(test_data$churned)
```

```
  0    1  
1067 918
```

```
table(master_new$churned)
```

```
  0    1  
3735 2881
```

11. STEPWISE GLM

```
clean_data <- train_data %>%  
  na.omit()  
  
null_model <- glm(  
  churned ~ 1,  
  data = clean_data,
```



```
family = binomial)
summary(null_model)
```

Call:

```
glm(formula = churned ~ 1, family = binomial, data = clean_data)
```

Coefficients:

```
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.61109    0.04066   15.03  <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for binomial family taken to be 1)

```
Null deviance: 3440  on 2651  degrees of freedom
Residual deviance: 3440  on 2651  degrees of freedom
AIC: 3442
```

Number of Fisher Scoring iterations: 4

```
full_model <- glm(
  churned ~ Employment + has_chronic_diagnosis + noshow_rate + age_range +
    Housing + Language_grouped + distance_meters,
  data = clean_data,
  family = binomial)

forward_model <- step(null_model,
  scope = formula(full_model),
  direction = "forward")
```

Start: AIC=3441.96

churned ~ 1

	Df	Deviance	AIC
+ Language_grouped	2	3360.9	3366.9
+ age_range	6	3367.9	3381.9
+ Employment	5	3389.0	3401.0
+ Housing	4	3393.8	3403.8
+ has_chronic_diagnosis	1	3408.8	3412.8
+ distance_meters	1	3434.0	3438.0
<none>		3440.0	3442.0
+ noshow_rate	1	3439.2	3443.2

Step: AIC=3366.88

churned ~ Language_grouped

	Df	Deviance	AIC
+ age_range	6	3291.9	3309.9
+ Employment	5	3306.2	3322.2

+ Housing	4	3331.5	3345.5
+ has_chronic_diagnosis	1	3346.6	3354.6
+ distance_meters	1	3353.8	3361.8
<none>		3360.9	3366.9
+ noshow_rate	1	3359.8	3367.8

Step: AIC=3309.88

churned ~ Language_grouped + age_range

	Df	Deviance	AIC
+ Employment	5	3257.9	3285.9
+ Housing	4	3266.6	3292.6
+ has_chronic_diagnosis	1	3278.0	3298.0
+ distance_meters	1	3285.3	3305.3
<none>		3291.9	3309.9
+ noshow_rate	1	3291.8	3311.8

Step: AIC=3285.87

churned ~ Language_grouped + age_range + Employment

	Df	Deviance	AIC
+ has_chronic_diagnosis	1	3244.6	3274.6
+ Housing	4	3242.2	3278.2
+ distance_meters	1	3251.8	3281.8
<none>		3257.9	3285.9
+ noshow_rate	1	3257.8	3287.8

Step: AIC=3274.65

churned ~ Language_grouped + age_range + Employment + has_chronic_diagnosis

	Df	Deviance	AIC
+ Housing	4	3230.6	3268.6
+ distance_meters	1	3238.4	3270.4
<none>		3244.6	3274.6
+ noshow_rate	1	3244.4	3276.4

Step: AIC=3268.6

churned ~ Language_grouped + age_range + Employment + has_chronic_diagnosis +
Housing

	Df	Deviance	AIC
+ distance_meters	1	3224.8	3264.8
<none>		3230.6	3268.6
+ noshow_rate	1	3230.2	3270.2

Step: AIC=3264.76

churned ~ Language_grouped + age_range + Employment + has_chronic_diagnosis +
Housing + distance_meters

	Df	Deviance	AIC
--	----	----------	-----

```
<none>          3224.8 3264.8
+ noshow_rate  1    3224.3 3266.3
```

```
backward_model <- step(null_model,
  scope = formula(full_model),
  direction = "backward")
```

Start: AIC=3441.96
churned ~ 1

```
stepwise_model <- step(null_model,
  scope = formula(full_model),
  direction = "both")
```

Start: AIC=3441.96
churned ~ 1

	Df	Deviance	AIC
+ Language_grouped	2	3360.9	3366.9
+ age_range	6	3367.9	3381.9
+ Employment	5	3389.0	3401.0
+ Housing	4	3393.8	3403.8
+ has_chronic_diagnosis	1	3408.8	3412.8
+ distance_meters	1	3434.0	3438.0
<none>		3440.0	3442.0
+ noshow_rate	1	3439.2	3443.2

Step: AIC=3366.88
churned ~ Language_grouped

	Df	Deviance	AIC
+ age_range	6	3291.9	3309.9
+ Employment	5	3306.2	3322.2
+ Housing	4	3331.5	3345.5
+ has_chronic_diagnosis	1	3346.6	3354.6
+ distance_meters	1	3353.8	3361.8
<none>		3360.9	3366.9
+ noshow_rate	1	3359.8	3367.8
- Language_grouped	2	3440.0	3442.0

Step: AIC=3309.88
churned ~ Language_grouped + age_range

	Df	Deviance	AIC
+ Employment	5	3257.9	3285.9
+ Housing	4	3266.6	3292.6
+ has_chronic_diagnosis	1	3278.0	3298.0
+ distance_meters	1	3285.3	3305.3
<none>		3291.9	3309.9

+ noshow_rate	1	3291.8	3311.8
- age_range	6	3360.9	3366.9
- Language_grouped	2	3367.9	3381.9

Step: AIC=3285.87

churned ~ Language_grouped + age_range + Employment

	Df	Deviance	AIC
+ has_chronic_diagnosis	1	3244.6	3274.6
+ Housing	4	3242.2	3278.2
+ distance_meters	1	3251.8	3281.8
<none>		3257.9	3285.9
+ noshow_rate	1	3257.8	3287.8
- Employment	5	3291.9	3309.9
- age_range	6	3306.2	3322.2
- Language_grouped	2	3338.5	3362.5

Step: AIC=3274.65

churned ~ Language_grouped + age_range + Employment + has_chronic_diagnosis

	Df	Deviance	AIC
+ Housing	4	3230.6	3268.6
+ distance_meters	1	3238.4	3270.4
<none>		3244.6	3274.6
+ noshow_rate	1	3244.4	3276.4
- has_chronic_diagnosis	1	3257.9	3285.9
- Employment	5	3278.0	3298.0
- age_range	6	3292.2	3310.2
- Language_grouped	2	3311.6	3337.6

Step: AIC=3268.6

churned ~ Language_grouped + age_range + Employment + has_chronic_diagnosis +
Housing

	Df	Deviance	AIC
+ distance_meters	1	3224.8	3264.8
<none>		3230.6	3268.6
+ noshow_rate	1	3230.2	3270.2
- Housing	4	3244.6	3274.6
- has_chronic_diagnosis	1	3242.2	3278.2
- Employment	5	3255.1	3283.1
- age_range	6	3278.3	3304.3
- Language_grouped	2	3288.5	3322.5

Step: AIC=3264.76

churned ~ Language_grouped + age_range + Employment + has_chronic_diagnosis +
Housing + distance_meters

	Df	Deviance	AIC
<none>		3224.8	3264.8
+ noshow_rate	1	3224.3	3266.3

- distance_meters	1	3230.6	3268.6
- Housing	4	3238.4	3270.4
- has_chronic_diagnosis	1	3236.6	3274.6
- Employment	5	3249.2	3279.2
- age_range	6	3271.9	3299.9
- Language_grouped	2	3283.3	3319.3

12. CROSS-VALIDATING LOGISTIC REGRESSION

```
set.seed(42)

cv_control <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = twoClassSummary
)

train_data$churned <- factor(
  train_data$churned,
  levels = c("0", "1"),
  labels = c("No_Churn", "Churn")
)

model_cv <- train(
  churned ~
    Employment +
    has_chronic_diagnosis +
    noshow_rate +
    Language_grouped +
    Housing +
    Education +
    distance_meters,
  data = train_data,
  na.action = na.omit,
  method = "glm",
  family = "binomial",
  trControl = cv_control,
  metric = "ROC"
)

# cross-val summary

print(model_cv)
```

Generalized Linear Model

4631 samples
7 predictor

2 classes: 'No_Churn', 'Churn'

No pre-processing

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 2470, 2470, 2470, 2470, 2469, 2470, ...

Resampling results:

ROC	Sens	Spec
0.6508805	0.1969989	0.9158876

```
# performance metrics (10-fold cv mean)
```

```
print(model_cv$results)
```

	parameter	ROC	Sens	Spec	ROCSD	SensSD	SpecSD
1	none	0.6508805	0.1969989	0.9158876	0.03470257	0.04102713	0.02864065

12. KNN MODEL

```
set.seed(100)
```

```
# What's our baseline accuracy if we just predict "Retained" for everyone?
```

```
# Note: calculating based on the majority class (0/Retained)
```

```
baseline_accuracy <- prop.table(table(master_new$churned))[1]
```

```
cat("Baseline accuracy (always predict 'Retained'):",  
    round(baseline_accuracy * 100, 2), "%\n")
```

Baseline accuracy (always predict 'Retained'): 56.45 %

```
master_new_clean <- na.omit(master_new)
```

```
# FIX: Convert to factor with string labels and force "Churned" (1) to be the  
#first level.
```

```
# This makes "Churned" the positive class by default.
```

```
master_new_clean$churned <- factor(master_new_clean$churned,  
                                   levels = c("1", "0"),  
                                   labels = c("Churned", "Retained"))
```

```
# Partitioning data
```

```
trainIndex <- createDataPartition(master_new_clean$churned, p = 0.8,  
                                   list = FALSE)
```

```
train_data <- master_new_clean[trainIndex, ]
```

```
test_data <- master_new_clean[-trainIndex, ]
```

```
# Training model
```

```
knndefault_acc <- train(  
  churned ~  
  Employment +
```

```

    age_range +
    Housing +
    Education +
    has_chronic_diagnosis +
    no_show_rate +
    distance_meters,
  data = train_data,
  method = "knn",
  preProcess = c("center", "scale")
)

print(knndefault_acc)

```

k-Nearest Neighbors

```

3070 samples
  7 predictor
  2 classes: 'Churned', 'Retained'

```

```

Pre-processing: centered (26), scaled (26)
Resampling: Bootstrapped (25 reps)
Summary of sample sizes: 3070, 3070, 3070, 3070, 3070, 3070, ...
Resampling results across tuning parameters:

```

k	Accuracy	Kappa
5	0.6213186	0.1335749
7	0.6273360	0.1352449
9	0.6269353	0.1247408

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was k = 7.

```

knn_predictions <- predict(knndefault_acc, newdata = test_data)

# FIX: Explicitly tell the matrix that "Churned" is the positive class
confusionMatrix(knn_predictions, test_data$churned, positive = "Churned")

```

Confusion Matrix and Statistics

	Reference	
Prediction	Churned	Retained
Churned	422	177
Retained	82	85

```

      Accuracy : 0.6619
      95% CI   : (0.6272, 0.6954)
No Information Rate : 0.658
P-Value [Acc > NIR] : 0.426

```

```

      Kappa : 0.1772

```

McNemar's Test P-Value : 5.193e-09

Sensitivity : 0.8373
Specificity : 0.3244
Pos Pred Value : 0.7045
Neg Pred Value : 0.5090
Prevalence : 0.6580
Detection Rate : 0.5509
Detection Prevalence : 0.7820
Balanced Accuracy : 0.5809

'Positive' Class : Churned

13. CROSS-VALIDATING kNN

```
set.seed(42)

# Grid of k values to test
k_grid <- expand.grid(k = seq(3, 21, by = 2))

# Define Control
# summaryFunction = twoClassSummary allows us to optimize for ROC (AUC)
# This REQUIRES the target variable to be a factor with text labels (e.g.,
#"Churned", "Retained")
cv_control <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = twoClassSummary
)

# NOTE: We assume train_data$churned is already properly formatted as
# "Churned" / "Retained" from the previous kNN model step.
# "Churned" should be the first level.

model_cv_knn <- train(
  churned ~
    Employment +
    age_range +
    Housing +
    Education +
    has_chronic_diagnosis +
    no_show_rate +
    distance_meters,
  data = train_data,
  method = "knn",
  trControl = cv_control,
```



```

tuneGrid = k_grid,
metric = "ROC", # Optimizing for Area Under Curve rather than just Accuracy
preProcess = c("center", "scale")
)

# kNN cross-val output (Check that Sens/Spec look correct for the "Churned" class)
print(model_cv_knn)

```

k-Nearest Neighbors

```

3070 samples
  7 predictor
  2 classes: 'Churned', 'Retained'

```

Pre-processing: centered (26), scaled (26)

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 2763, 2763, 2763, 2763, 2763, 2763, ...

Resampling results across tuning parameters:

k	ROC	Sens	Spec
3	0.6208534	0.7702970	0.4085714
5	0.6221829	0.7881188	0.3780952
7	0.6265912	0.8054455	0.3380952
9	0.6323527	0.8193069	0.3219048
11	0.6301485	0.8381188	0.3095238
13	0.6332815	0.8415842	0.2904762
15	0.6351532	0.8529703	0.2790476
17	0.6362636	0.8643564	0.2628571
19	0.6345191	0.8643564	0.2571429
21	0.6351320	0.8717822	0.2476190

ROC was used to select the optimal model using the largest value.

The final value used for the model was k = 17.

```

# The best k value found:
print(model_cv_knn$bestTune)

```

```

k
8 17

```

```

# Predict using the best model
# type = "prob" gives the probability scores (needed for ROC curves)
predictions_prob <- predict(model_cv_knn, newdata = test_data, type = "prob")

# type = "raw" gives the actual class labels
predictions_class <- predict(model_cv_knn, newdata = test_data, type = "raw")

# Final Confusion Matrix on Test Data

```

```
# positive = "Churned" ensures we are focusing on the target class correctly
confusionMatrix(predictions_class, test_data$churned, positive = "Churned")
```

Confusion Matrix and Statistics

	Reference	
Prediction	Churned	Retained

Churned	432	196
---------	-----	-----

Retained	72	66
----------	----	----

Accuracy : 0.6501

95% CI : (0.6152, 0.6839)

No Information Rate : 0.658

P-Value [Acc > NIR] : 0.6908

Kappa : 0.123

McNemar's Test P-Value : 5.76e-14

Sensitivity : 0.8571

Specificity : 0.2519

Pos Pred Value : 0.6879

Neg Pred Value : 0.4783

Prevalence : 0.6580

Detection Rate : 0.5640

Detection Prevalence : 0.8198

Balanced Accuracy : 0.5545

'Positive' Class : Churned

14. LDA

```
# IMPORTANT: Ensure 'churned' is a factor before running the check
# If your 'churned' variable is not already a factor (e.g., if it's 0/1 integer),
# uncomment and run:
# train_data$churned <- as.factor(train_data$churned)

# List of categorical features from your model
cat_features <- c("Employment", "age_range", "Housing", "Language_grouped",
                 "has_chronic_diagnosis")

# Fixed function to check and print group counts using base R indexing for
# robustness
check_lda_groups_fix <- function(df, feature) {
  cat("\n--- Checking Feature:", feature, "---\n")

  # Group, count, and pivot to wide format
  df_summary <- df %>%
```

```

group_by(churned, .data[[feature]]) %>%
summarise(n = n(), .groups = 'drop') %>%
pivot_wider(names_from = churned, values_from = n, values_fill = 0)

# Standardize column names
names(df_summary)[names(df_summary) == "Retained"] <- "Not_Churned"
names(df_summary)[names(df_summary) == "Churned"] <- "Churned"

df_summary <- df_summary %>%
  mutate(
    Total = Not_Churned + Churned,
    Churn_Rate = round(Churned / Total, 3)
  )

# Simplified final selection using base R indexing [ ] to avoid dplyr
#compatibility issues
df_final <- df_summary[, c(feature, "Not_Churned", "Churned", "Total",
                           "Churn_Rate")]

print(df_final, n=Inf)
}

# Run the check for all key categorical features
lapply(cat_features, check_lda_groups_fix, df = train_data)

```

--- Checking Feature: Employment ---

A tibble: 6 × 5

Employment	Not_Churned	Churned	Total	Churn_Rate
<fct>	<int>	<int>	<int>	<dbl>
1 Employed	718	1094	1812	0.604
2 Self Employed	58	144	202	0.713
3 Seasonal	16	25	41	0.61
4 Unemployed (Actively Looking For Work)	200	613	813	0.754
5 Not In Workforce (Choosing Not To Work)	38	61	99	0.616
6 Retired	20	83	103	0.806

--- Checking Feature: age_range ---

A tibble: 7 × 5

age_range	Not_Churned	Churned	Total	Churn_Rate
<fct>	<int>	<int>	<int>	<dbl>
1 18-19	8	3	11	0.273
2 20-29	149	218	367	0.594
3 30-39	237	408	645	0.633
4 40-49	251	402	653	0.616
5 50-59	210	393	603	0.652
6 60-64	122	238	360	0.661
7 65+	73	358	431	0.831

--- Checking Feature: Housing ---

A tibble: 5 × 5

	Housing	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>
1	Own	158	280	438	0.639
2	Rent	601	1029	1630	0.631
3	Lives With Others	233	652	885	0.737
4	Mobile Home	50	34	84	0.405
5	Homeless	8	25	33	0.758

--- Checking Feature: Language_grouped ---

A tibble: 3 × 5

	Language_grouped	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>
1	English	684	1598	2282	0.7
2	Spanish	316	357	673	0.53
3	Other	50	65	115	0.565

--- Checking Feature: has_chronic_diagnosis ---

A tibble: 2 × 5

	has_chronic_diagnosis	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>
1	0	606	1382	1988	0.695
2	1	444	638	1082	0.59

[[1]]

A tibble: 6 × 5

	Employment	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>
1	Employed	718	1094	1812	0.604
2	Self Employed	58	144	202	0.713
3	Seasonal	16	25	41	0.61
4	Unemployed (Actively Looking For Work)	200	613	813	0.754
5	Not In Workforce (Choosing Not To Work)	38	61	99	0.616
6	Retired	20	83	103	0.806

[[2]]

A tibble: 7 × 5

	age_range	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>
1	18-19	8	3	11	0.273
2	20-29	149	218	367	0.594
3	30-39	237	408	645	0.633
4	40-49	251	402	653	0.616
5	50-59	210	393	603	0.652
6	60-64	122	238	360	0.661
7	65+	73	358	431	0.831

[[3]]

A tibble: 5 × 5

	Housing	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>

1 Own	158	280	438	0.639
2 Rent	601	1029	1630	0.631
3 Lives With Others	233	652	885	0.737
4 Mobile Home	50	34	84	0.405
5 Homeless	8	25	33	0.758

```
[[4]]
```

```
# A tibble: 3 × 5
```

	Language_grouped	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>
1	English	684	1598	2282	0.7
2	Spanish	316	357	673	0.53
3	Other	50	65	115	0.565

```
[[5]]
```

```
# A tibble: 2 × 5
```

	has_chronic_diagnosis	Not_Churned	Churned	Total	Churn_Rate
	<fct>	<int>	<int>	<int>	<dbl>
1	0	606	1382	1988	0.695
2	1	444	638	1082	0.59

14.1: Recode Sparse Categories to Fix Instability

```
train_data_recoded <- train_data %>%
  # Recode age_range 18-19 into 20-29
  mutate(age_range_new = case_when(
    age_range == "18-19" ~ "20-29",
    TRUE ~ as.character(age_range)
  )) %>%

  # Recode Housing Homeless into Lives With Others
  mutate(Housing_new = case_when(
    Housing == "Homeless" ~ "Lives With Others",
    TRUE ~ as.character(Housing)
  )) %>%

  # Recode Employment Seasonal into Unemployed
  mutate(Employment_new = case_when(
    Employment == "Seasonal" ~ "Unemployed (Actively Looking For Work)",
    TRUE ~ as.character(Employment)
  )) %>%

  # Convert new variables back to factors (critical for LDA)
  mutate(
    age_range_new = factor(age_range_new),
    Housing_new = factor(Housing_new),
    Employment_new = factor(Employment_new),
    churned = as.factor(churned)
  )
```

```
train_data_recoded <- train_data_recoded %>%
  mutate(Language_grouped = droplevels(Language_grouped))
```

14.2: Fit the LDA Model with Recoded Factors

```
churn_lda_final_sdoH <- lda(
  churned ~ Employment_new +
    has_chronic_diagnosis +
    age_range_new +
    Housing_new +
    Language_grouped,
  data = train_data_recoded,
) # Final Model: SDoH-Only (Removing both continuous variables)

# Print the LD1 coefficients
print(churn_lda_final_sdoH)
```

Call:

```
lda(churned ~ Employment_new + has_chronic_diagnosis + age_range_new +
  Housing_new + Language_grouped, data = train_data_recoded,
)
```

Prior probabilities of groups:

```
Churned Retained
0.6579805 0.3420195
```

Group means:

	Employment_newNot In Workforce (Choosing Not To Work)		
Churned	0.03019802		
Retained	0.03619048		
	Employment_newRetired	Employment_newSelf Employed	
Churned	0.04108911	0.07128713	
Retained	0.01904762	0.05523810	
	Employment_newUnemployed (Actively Looking For Work)		
Churned	0.3158416		
Retained	0.2057143		
	has_chronic_diagnosis1	age_range_new30-39	age_range_new40-49
Churned	0.3158416	0.2019802	0.1990099
Retained	0.4228571	0.2257143	0.2390476
	age_range_new50-59	age_range_new60-64	age_range_new65+
Churned	0.1945545	0.1178218	0.17722772
Retained	0.2000000	0.1161905	0.06952381
	Housing_newMobile Home	Housing_newOwn	Housing_newRent
Churned	0.01683168	0.1386139	0.5094059
Retained	0.04761905	0.1504762	0.5723810
	Language_groupedSpanish	Language_groupedOther	

Churned	0.1767327	0.03217822
Retained	0.3009524	0.04761905

Coefficients of linear discriminants:

	LD1
Employment_newNot In Workforce (Choosing Not To Work)	-0.4449702
Employment_newRetired	-0.7994912
Employment_newSelf Employed	-0.9693034
Employment_newUnemployed (Actively Looking For Work)	-0.8617720
has_chronic_diagnosis1	0.5383409
age_range_new30-39	-0.5513480
age_range_new40-49	-0.5912245
age_range_new50-59	-0.7182459
age_range_new60-64	-0.4483067
age_range_new65+	-1.7510939
Housing_newMobile Home	1.6525029
Housing_newOwn	0.7606826
Housing_newRent	0.4939473
Language_groupedSpanish	1.1150070
Language_groupedOther	1.4849567

14.3: LDA Coefficient Interpretation

Based on the LD1 output from our linear discriminant model, the strongest predictors for Churning (indicated by negative coefficients) are age and employment status. Specifically, being in the 65+ age range is the single most significant driver of churn (-1.75), followed by Self-Employed (-0.97) and Unemployed (-0.86). This suggests that older individuals and those with less stable or traditional employment are far more likely leave Lackey. Conversely, the strongest predictors for Retention (indicated by positive coefficients) are related to housing and language. Living in a Mobile Home ($+1.65$) is the most powerful indicator of retention, along with speaking non-English languages (Other at $+1.48$ and Spanish at $+1.12$).

15. VISUALS

```
master_new <- master_new %>%
  mutate(
    # Create 3-month enrollment bins
    enroll_bin = cut(
      last_enrollment_end,
      breaks = seq(
        from = floor_date(min(last_enrollment_end, na.rm = TRUE),
                           unit = "month"),
        to = ceiling_date(max(last_enrollment_end, na.rm = TRUE),
                           unit = "month") + months(4),
        by = "4 months"
      ),
    ),
    include_lowest = TRUE,
```

```

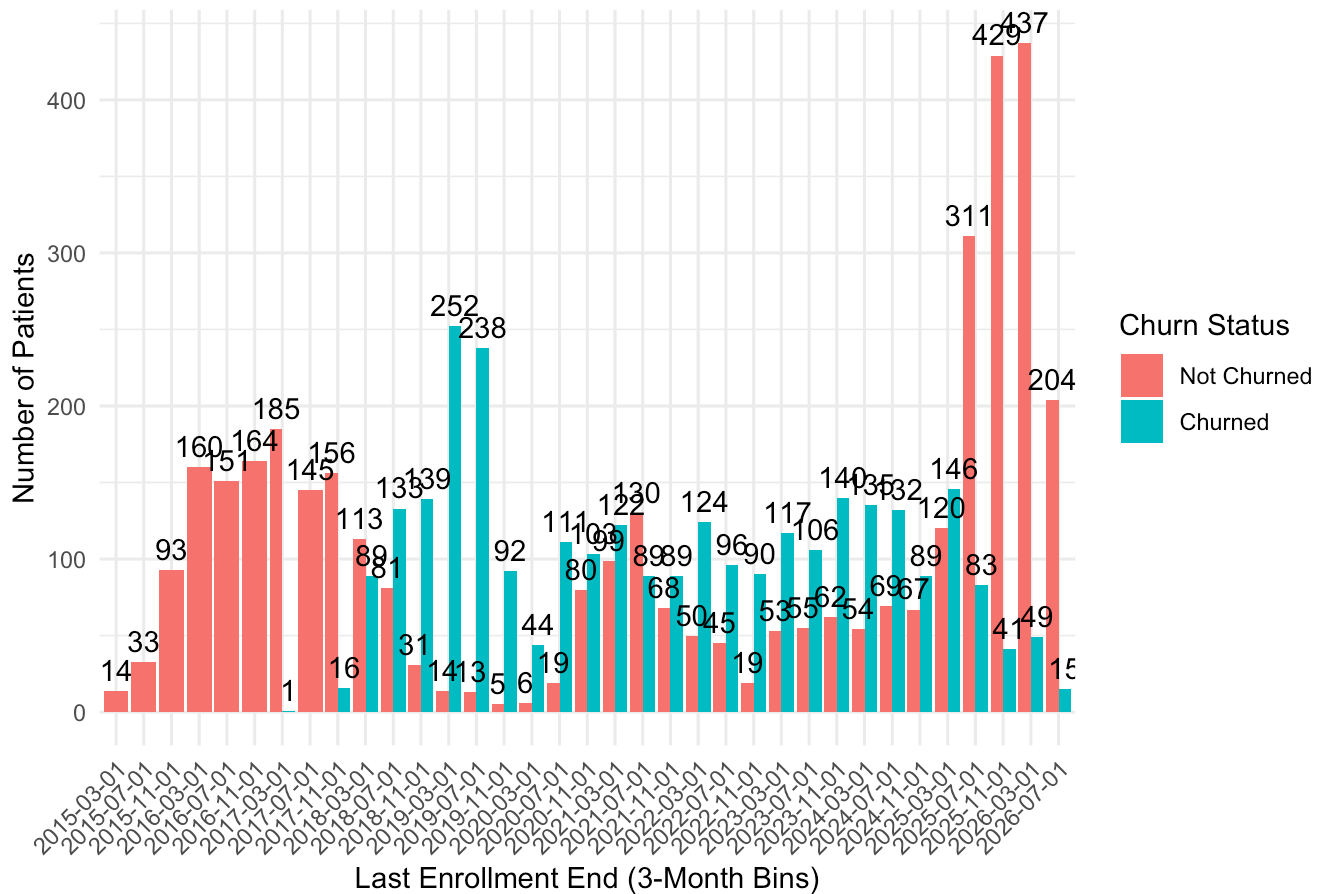
    right = FALSE
  ),
  # Create churn label, convert NA to "NA" string
  churned_label = case_when(
    churned == 1 ~ "Churned",
    churned == 0 ~ "Not Churned",
    is.na(churned) ~ "NA"
  ),
  churned_label = factor(churned_label,
                        levels = c("Not Churned", "Churned", "NA"))
)

enroll_counts <- master_new %>%
  group_by(enroll_bin, churned_label) %>%
  summarise(n = n(), .groups = "drop")

ggplot(enroll_counts, aes(x = enroll_bin, y = n, fill = churned_label)) +
  geom_col(position = "dodge") +
  geom_text(aes(label = n),
            position = position_dodge(width = 0.9), vjust = -0.5) +
  labs(
    title = "Patients by Last Enrollment End (3-Month Bins) and Churn Status",
    x = "Last Enrollment End (3-Month Bins)",
    y = "Number of Patients",
    fill = "Churn Status"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```


Patients by Last Enrollment End (3-Month Bins) and Churn Status



```
table(master_new$churned)
```

```

  0    1
3735 2881

```

```

library(ggplot2)
library(dplyr)
library(scales)

```

Warning: package 'scales' was built under R version 4.3.3

Attaching package: 'scales'

The following object is masked from 'package:purrr':

discard

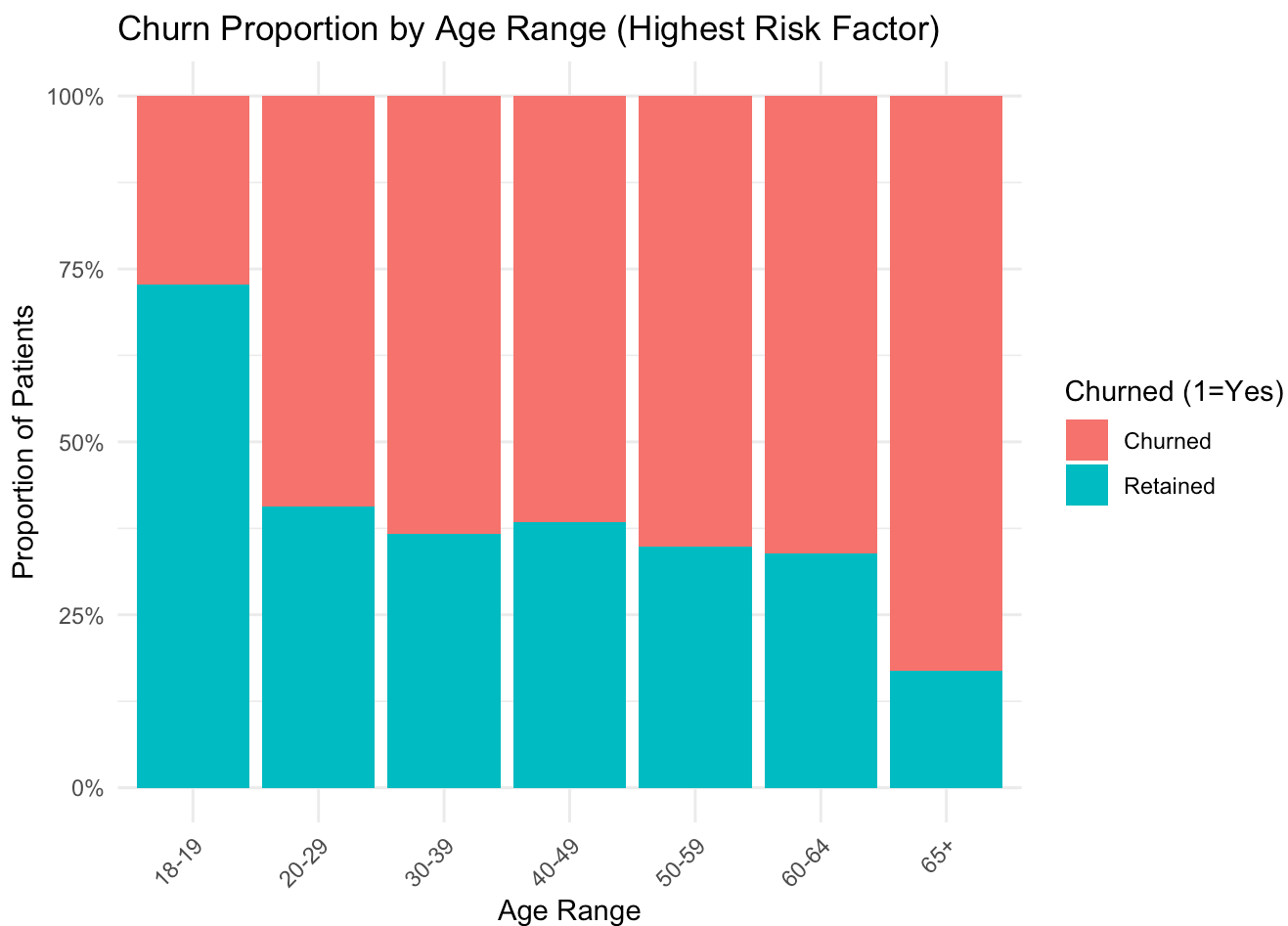
The following object is masked from 'package:readr':

col_factor

```
# Visualization of Churn Breakdown by our Churn Def

# Age Range Plot
train_data %>%
  # Ensure age_range is ordered logically (optional, but good practice)
  mutate(age_range = factor(age_range, levels = c("18-19", "20-29", "30-39",
                                                  "40-49", "50-59", "60-64",
                                                  "65+"))) %>%

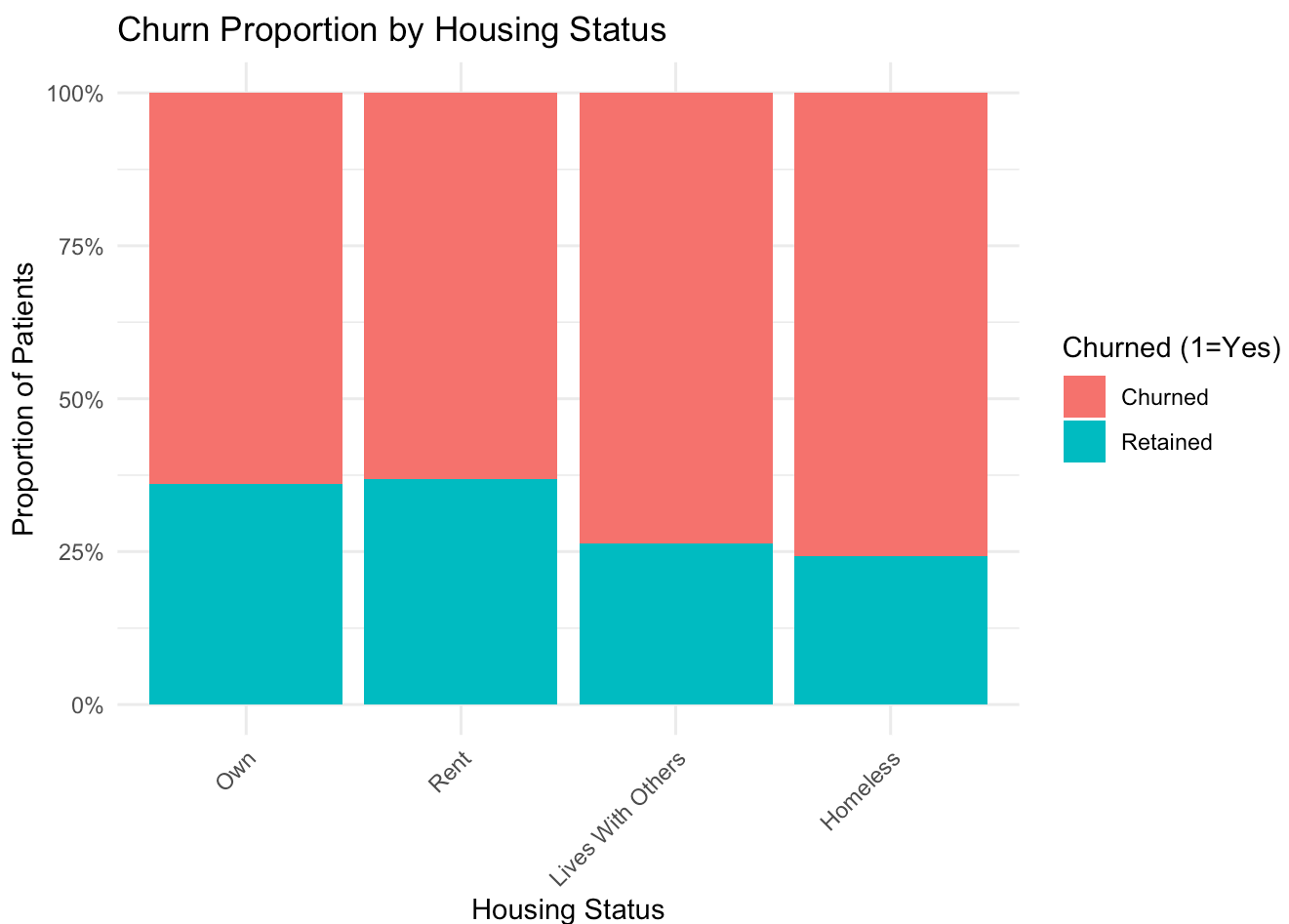
  ggplot(aes(x = age_range, fill = factor(churned))) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = percent) +
  labs(
    title = "Churn Proportion by Age Range (Highest Risk Factor)",
    y = "Proportion of Patients",
    x = "Age Range",
    fill = "Churned (1=Yes)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
# Housing Status Plot
train_data %>%
```

```
# Filter out low-impact or non-significant housing groups if desired, or keep all
filter(Housing %in% c("Own", "Rent", "Lives With Others", "Homeless")) %>%

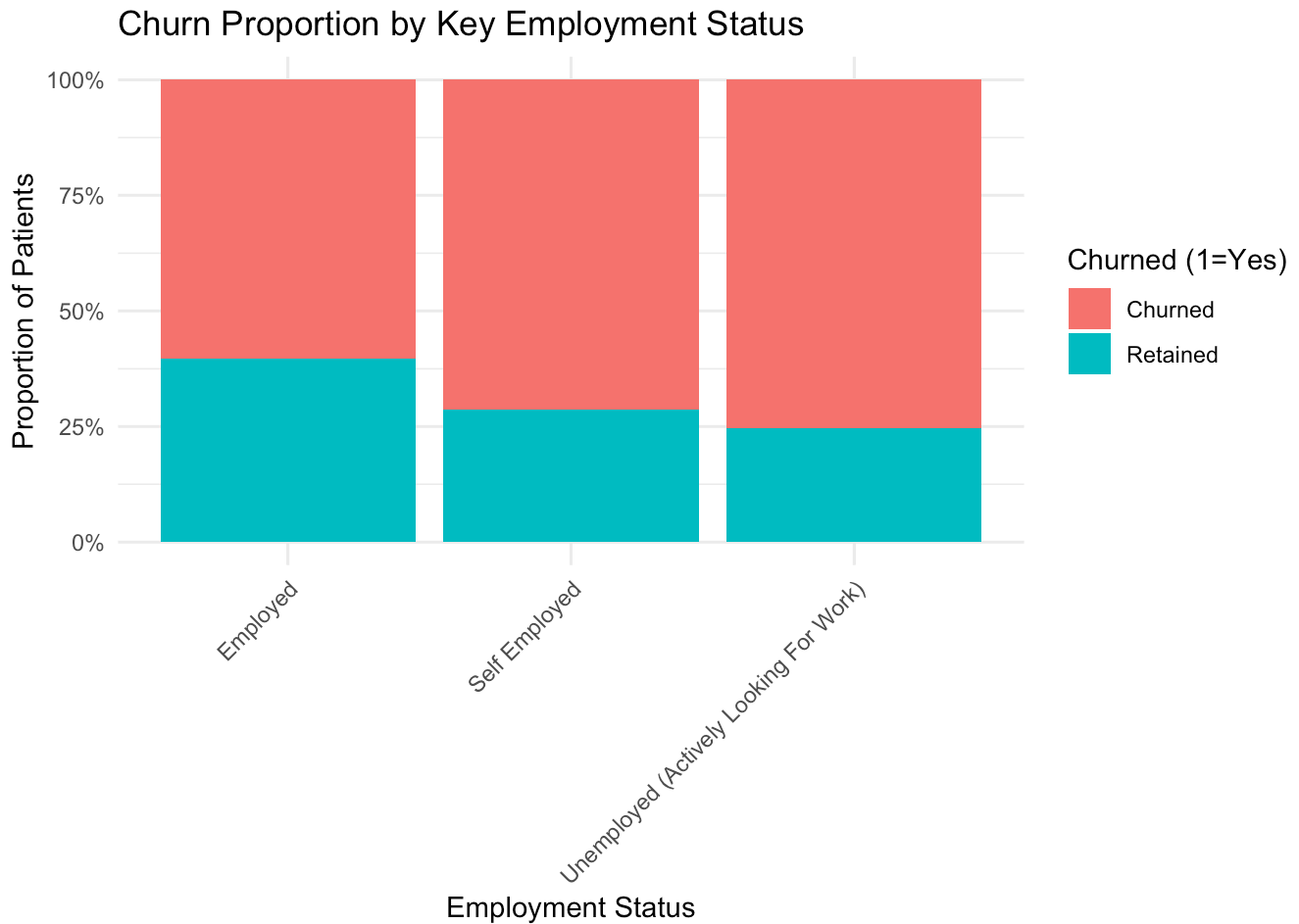
ggplot(aes(x = Housing, fill = factor(churned))) +
geom_bar(position = "fill") +
scale_y_continuous(labels = percent) +
labs(
  title = "Churn Proportion by Housing Status",
  y = "Proportion of Patients",
  x = "Housing Status",
  fill = "Churned (1=Yes)"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
# Employment Status Plot
train_data %>%
  # Filter to focus on the significant groups (Employed is reference)
  filter(Employment %in% c("Employed", "Self Employed",
    "Unemployed (Actively Looking For Work)")) %>%

  ggplot(aes(x = Employment, fill = factor(churned))) +
  geom_bar(position = "fill") +
```

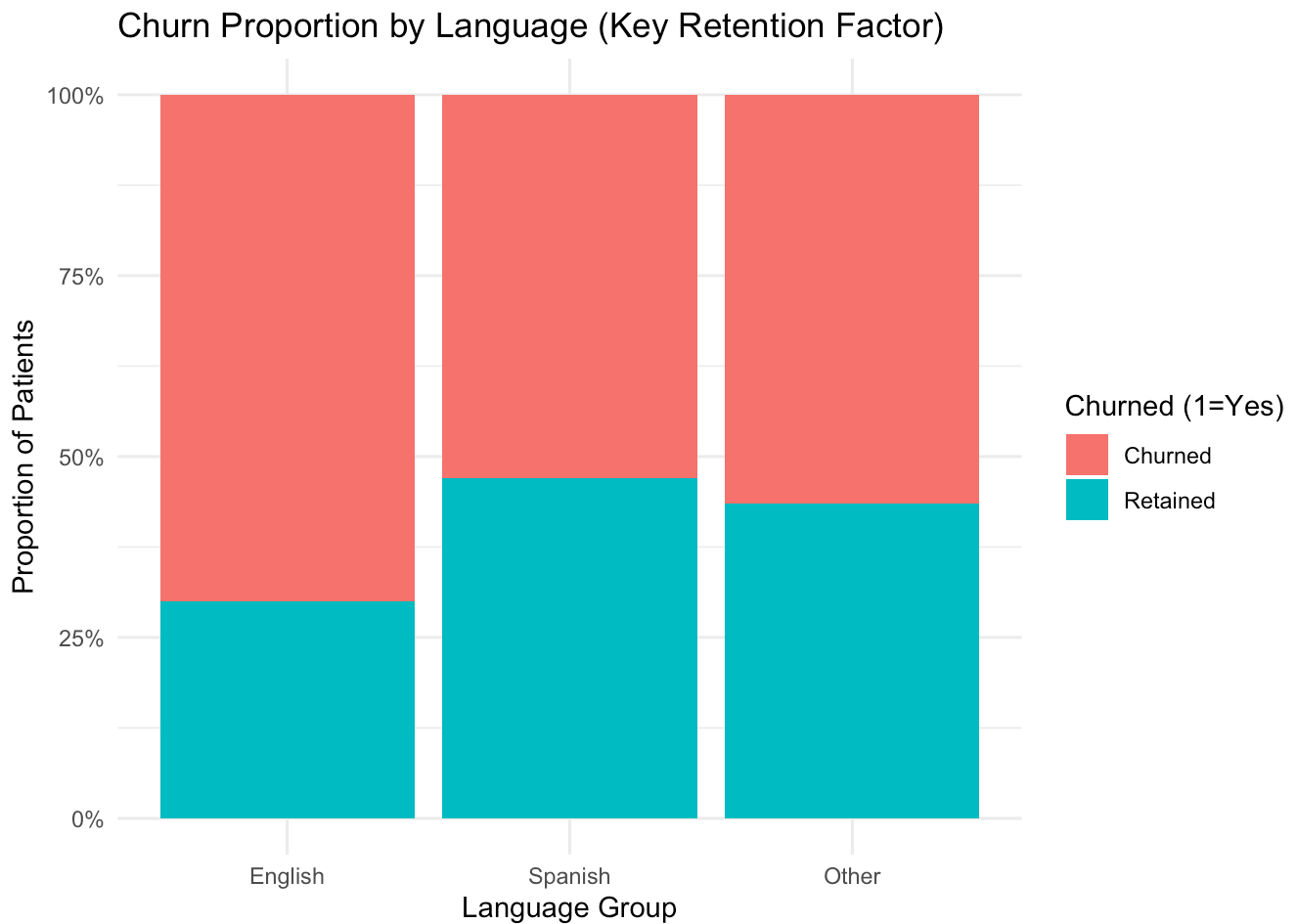
```
scale_y_continuous(labels = percent) +
labs(
  title = "Churn Proportion by Key Employment Status",
  y = "Proportion of Patients",
  x = "Employment Status",
  fill = "Churned (1=Yes)"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
# Language Plot
train_data %>%
  # Filter to focus on the significant groups
  filter(Language_grouped %in% c("English", "Spanish", "Other")) %>%

ggplot(aes(x = Language_grouped, fill = factor(churned))) +
geom_bar(position = "fill") +
scale_y_continuous(labels = percent) +
labs(
  title = "Churn Proportion by Language (Key Retention Factor)",
  y = "Proportion of Patients",
  x = "Language Group",
  fill = "Churned (1=Yes)"
)
```

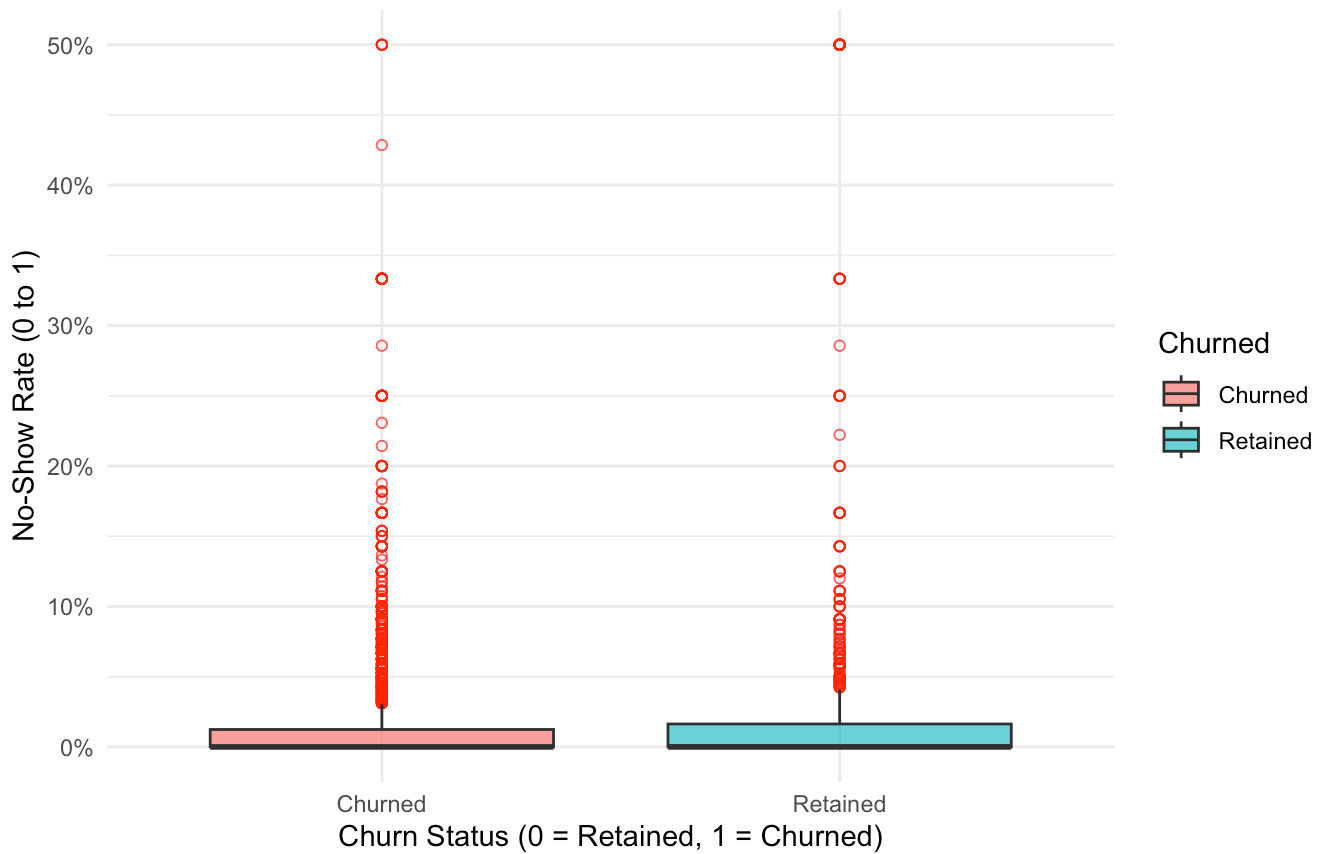
```
) +  
theme_minimal()
```



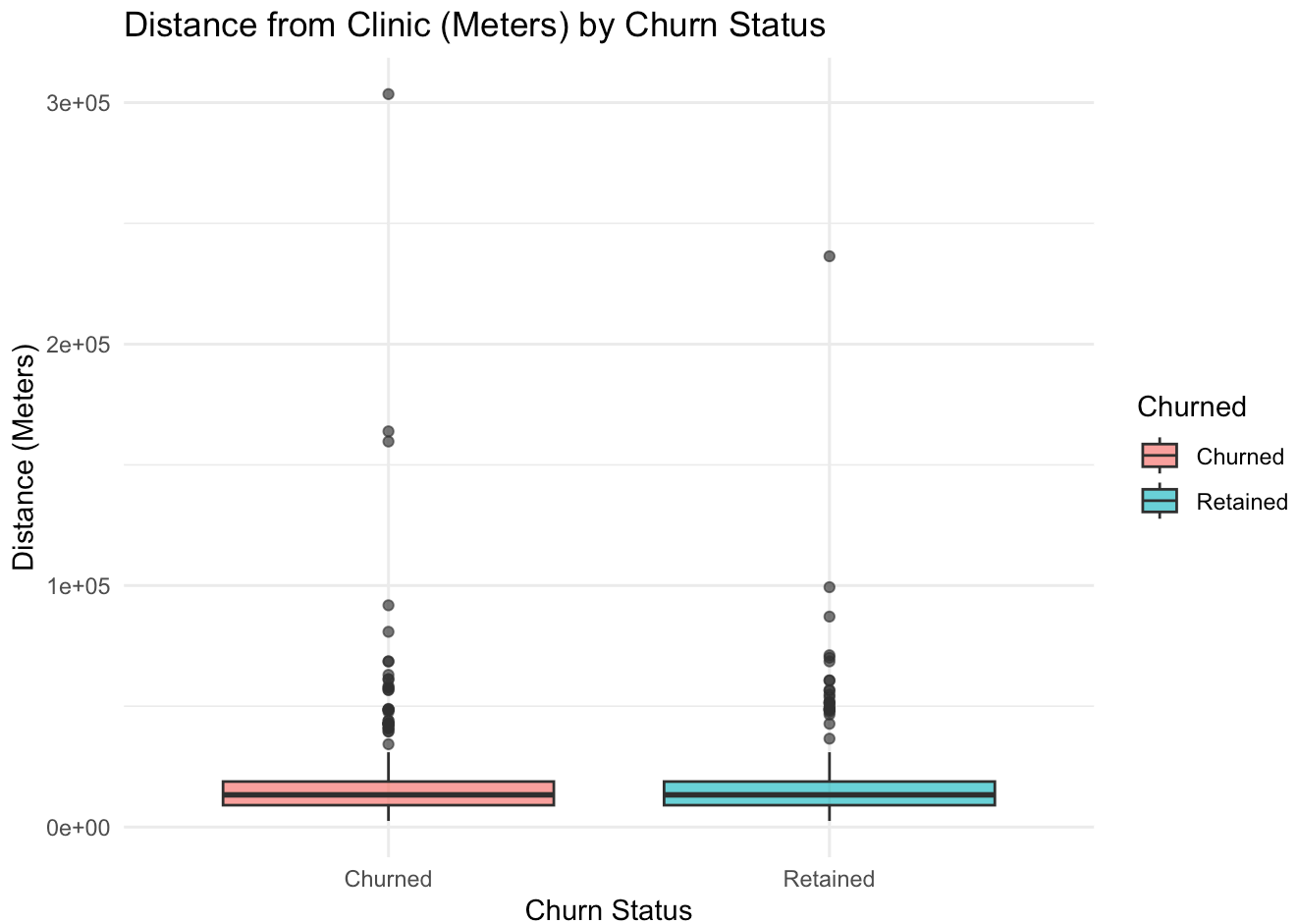
```
# No-Show Rate vs Churn  
ggplot(train_data, aes(x = factor(churned), y = noshow_rate,  
                        fill = factor(churned))) +  
geom_boxplot(alpha = 0.7, outlier.colour = "red", outlier.shape = 1) +  
labs(  
  title = "Distribution of No-Show Rates by Churn Status",  
  subtitle = "Do patients who miss appointments churn more often?",  
  x = "Churn Status (0 = Retained, 1 = Churned)",  
  y = "No-Show Rate (0 to 1)",  
  fill = "Churned"  
) +  
theme_minimal() +  
scale_y_continuous(labels = scales::percent) # Formats Y axis as percentage
```

Distribution of No-Show Rates by Churn Status

Do patients who miss appointments churn more often?



```
# Distance vs Churn
ggplot(train_data, aes(x = factor(churned), y = distance_meters,
                        fill = factor(churned))) +
  geom_boxplot(alpha = 0.7) +
  # scale_y_log10() + # Uncomment this line if outliers make the plot hard to read
  labs(
    title = "Distance from Clinic (Meters) by Churn Status",
    x = "Churn Status",
    y = "Distance (Meters)",
    fill = "Churned"
  ) +
  theme_minimal()
```



```
# Chronic Diagnosis vs Churn
train_data %>%
  # Filter out NAs if necessary, though your cleaning steps likely handled this
  filter(!is.na(has_chronic_diagnosis)) %>%

  ggplot(aes(x = factor(has_chronic_diagnosis), fill = factor(churned))) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  scale_x_discrete(labels = c("0" = "No Chronic Condition",
                              "1" = "Has Chronic Condition")) +
  labs(
    title = "Churn Proportion by Chronic Condition Status",
    subtitle = "Patients with chronic conditions are often 'stickier' (lower churn)",
    y = "Proportion of Patients",
    x = "Diagnosis Status",
    fill = "Churned (1=Yes)"
  ) +
  theme_minimal()
```

Churn Proportion by Chronic Condition Status

Patients with chronic conditions are often 'stickier' (lower churn)

